

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : R.Y.Thrayamabaha Balajee

Reg. No : 11249A381

Class : II B.E. (CSE)

Section : S4

Course Code : BCF244AI2

Course Name : Data Science Tool kit Lab

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by
Mr/Ms. R.Y.Thrayambaha Balajee,

with Reg. No 11249A381 of II-B.E.(CSE) during the academic
year 2025 – 2026.

Station: Enathur

Date: 26.04.2026

Staff-in-charge

Head of the Department

Submitted for the Practical examination held on 02.05.2026.

Examiner-1

Examiner-2

INDEX

SL.NO	Name of the experiment	Date	Page no	Signature
1	Import Data from CSV, Excel, JSON, etc. using Pandas, Matplotlib, Seaborn, Sklearn	19.01.2026		
2	Perform Data exploration and preprocessing in Python	02.02.2026		
3	Implement regularised Linear regression	09.02.2026		
4	Implement Naive Bayes classifier	16.02.2026		
5	Implement regularized logistic regression	02.03.2026		
6	Build models using different Ensembling techniques	09.03.2026		
7	Build models using Decision trees	16.03.2026		
8	Build model using SVM with different kernels	06.04.2026		
9	Implement K-NN algorithm to classify a dataset.	06.04.2026		
10	Build model to perform Clustering using K-means after applying PCA and determining the value of K using Elbow method	13.04.2026		

1.1.numpy package

Aim:

To study and understand the basic functionalities of Python libraries such as NumPy for numerical operations, Pandas for data manipulation, Matplotlib for visualization, and Scikit-learn for machine learning.

#1 array creation

```
import numpy as np
a=np.array([1,2,3,4,5])
b=np.arange(1,10,2)
c=np.linspace(0,10,5)
d=np.zeros((3,3))
e=np.ones((3,3))
f=np.eye(3)
g=np.random.randint(1,100,size=(3,3))
print("Array a:",a)
print("Arrange b:",b)
print("Linspace c:",c)
print("Zeros d:",d)
print("Ones e:",e)
print("Identity matrix f:",f)
print("Random matrix g:",g)
```

```
Array a: [1 2 3 4 5]
Arrange b: [1 3 5 7 9]
Linspace c: [ 0.   2.5  5.   7.5 10. ]
Zeros d: [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Ones e: [[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
Identity matrix f: [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random matrix g: [[19 85 50]
 [75 14  1]
 [98 91 93]]
```

#2 array properties

```
'''Random matrix g: [[98 19 69]
                    At  [25 36 30]
                    [ 2 58 72]]'''
print("\nShape:",g.shape)
print("Size:",g.size)
print("Data type:",g.dtype)
```

```
print("Dimension:",g.ndim)
print("Item size:",g.itemsize)
```

```
Shape: (3, 3)
Size: 9
Data type: int64
Dimension: 2
Item size: 8
```

```
#3 reshape & transpose
'''Random matrix g: [[98 19 69]
                    [25 36 30]
                    [ 2 58 72]]'''
```

```
reshaped=g.reshape(1,9)
print("\nReshaped Array:",reshaped)
print("Transpose:\n",g.T)
print("Flattened:",g.flatten())
```

```
Reshaped Array: [[19 85 50 75 14  1 98 91 93]]
```

```
Transpose:
```

```
[[19 75 98]
 [85 14 91]
 [50  1 93]]
```

```
Flattened: [19 85 50 75 14  1 98 91 93]
```

```
#4 Mathematical Opertions
```

```
#Array a: [1 2 3 4 5]
```

```
print("multiply:",np.multiply(a,2))
print("square root:",np.sqrt(a))
print("power:",np.power(a,2))
print("absolute:",np.abs([-1,-2,3]))
```

```
print("Square:",np.square(a))
print("\nAdd:",np.add(a,2))
print("Subtract:",np.subtract(a,2))
print("Divide:",np.divide(a,2))
```

```
multiply: [ 2  4  6  8 10]
```

```
square root: [1.          1.41421356  1.73205081  2.          2.23606798]
```

```
power: [ 1  4  9 16 25]
```

```
absolute: [1 2 3]
```

```
Square: [ 1  4  9 16 25]
```

```
Add: [3 4 5 6 7]
```

```
Subtract: [-1  0  1  2  3]
```

```
Divide: [0.5 1.  1.5 2.  2.5]
```

```
#5 statistical methods
```

```
#Array a: [1 2 3 4 5]
```

```

print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Min:", np.min(a))
print("Max:", np.max(a))
print("Sum:", np.sum(a))
print("percentile(20)", np.percentile(a, 20))

```

```

Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Variance: 2.0
Min: 1
Max: 5
Sum: 15
percentile(20) 1.8

```

```

#7 indexing and slicing
#Array a: [1 2 3 4 5]
print("\nfirst element:", a[0])
print("slice:", a[1:4])
'''Random matrix g: [[98 19 69]
                    [25 36 30]
                    [ 2 58 72]]'''
print("2D slice:", g[0:2, 1:3])

```

```

first element: 1
slice: [2 3 4]
2D slice: [[85 50]
          [14  1]]

```

```

#8 boolean & conditional
'''Random matrix g: [[98 19 69]
                    [25 36 30]
                    [ 2 58 72]]'''
print("\nwhere > 50:", np.where(g>50))
print("any > 50:", np.any(g>50))
print("all > 50:", np.all(g>50))

```

```

where > 50: (array([0, 1, 2, 2, 2]), array([1, 0, 0, 1, 2]))
any > 50: True
all > 50: False

```

```

#9 sorting & searching
#Array a: [1 2 3 4 5]
print("\n sorted array a:", np.sort(a))
print("index of max a:", np.argmax(a))

```

```
print("index of min a:",np.argmin(a))
print("unique elements:",np.unique([1,2,3,4,4]))
```

```
sorted array a: [1 2 3 4 5]
index of max a: 4
index of min a: 0
unique elements: [1 2 3 4]
```

#10 linear algebra

```
x=np.array([[1,2],[3,4]])
y=np.array([[5,6],[7,8]])
print("\nDot product:\n",np.dot(x,y))
print("Inverse:\n",np.linalg.inv(x))
```

Dot product:

```
[[19 22]
```

```
[43 50]]
```

Inverse:

```
[[-2.  1. ]
```

```
[ 1.5 -0.5]]
```

1.2.pandas

A) Creating Dataframes

```
import pandas as pd
#from a dictionary
data={
    'name':['alice','bob','charlie'],
    'age':[25,30,35],
    'city':['new york','san francisco','los angeles']
}
df=pd.DataFrame(data)
print(df)
#viewing data
print(df.head())
print(df.info())
print(df.describe())
```

```
   name  age      city
0  alice   25  new york
1   bob   30  san francisco
2 charlie   35  los angeles
   name  age      city
0  alice   25  new york
1   bob   30  san francisco
2 charlie   35  los angeles
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0    name    3 non-null      object
1    age     3 non-null      int64
2    city    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
      age
count  3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
```

B) Selection And Filtering

```
#selection and filtering
ages=df['age']
#Filtering rows
above_25=df[df['age']>25]
#iloc(integer_based) vs loc(label_based)
row_0=df.iloc[0]#first row(Alice's data)
specific_val=df.loc[0,'city']#'new york'
print(above_25)
```

	name	age	city
1	bob	30	san francisco
2	charlie	35	los angeles

C)Data cleaning(Handling missing values)

```
#checking for nulls
print(df.isnull().sum())
#dropping missing values
df_cleaned=df.dropna()
#filling missing values
df_filled=df.fillna(0)#fill NaNs with 0
#or fill with mean
df['age']=df['age'].fillna(df['age'].mean())
print(df['age'])
```

```
name      0
age       0
city      0
dtype: int64
0      25
1      30
2      35
Name: age, dtype: int64
```

Student management system scenario based questions

Codes after importing CSV file(dataset for pandas(11249a381) - sheet1.csv)

```
import pandas as pd
df=pd.read_csv('/content/dataset for pandas(11249a381) - Sheet1.csv')
print(df)
```

	Roll.no	Name	Age	Marks	Department	Section
0	1	Alice	20	78.0	CSE	A
1	2	Bob	21	NaN	ECE	B
2	3	Charlie	19	65.0	CSE	A
3	4	David	22	45.0	EEE	C
4	5	Eva	20	88.0	CSE	B
5	6	Bob	21	55.0	ECE	B

```
import pandas as pd
df=pd.read_csv('/content/dataset for pandas(11249a381) - Sheet1.csv')
print(df)
```

	Roll.no	Name	Age	Marks	Department	Section
0	1	Alice	20	78.0	CSE	A
1	2	Bob	21	NaN	ECE	B
2	3	Charlie	19	65.0	CSE	A
3	4	David	22	45.0	EEE	C
4	5	Eva	20	88.0	CSE	B
5	6	Bob	21	55.0	ECE	B

```
import pandas as pd
df=pd.read_csv('/content/dataset for pandas(11249a381) - Sheet1.csv')
print(df)
```

	Roll.no	Name	Age	Marks	Department	Section
0	1	Alice	20	78.0	CSE	A
1	2	Bob	21	NaN	ECE	B
2	3	Charlie	19	65.0	CSE	A
3	4	David	22	45.0	EEE	C
4	5	Eva	20	88.0	CSE	B
5	6	Bob	21	55.0	ECE	B

```
import pandas as pd
df=pd.read_csv('/content/dataset for pandas(11249a381) - Sheet1.csv')
print(df)
```

	Roll.no	Name	Age	Marks	Department	Section
0	1	Alice	20	78.0	CSE	A
1	2	Bob	21	NaN	ECE	B
2	3	Charlie	19	65.0	CSE	A
3	4	David	22	45.0	EEE	C
4	5	Eva	20	88.0	CSE	B
5	6	Bob	21	55.0	ECE	B

```
#2 display only name and marks
print(df[['Name', 'Marks']])
```

	Name	Marks
0	Alice	78.0
1	Bob	NaN
2	Charlie	65.0

```
3    David    45.0
4      Eva    88.0
5      Bob    55.0
```

#3 some students have missing values in the marks column.replace with 0

```
df['Marks']=df['Marks'].fillna(0)
print(df)
```

	Roll.no	Name	Age	Marks	Department	Section
0	1	Alice	20	78.0	CSE	A
1	2	Bob	21	0.0	ECE	B
2	3	Charlie	19	65.0	CSE	A
3	4	David	22	45.0	EEE	C
4	5	Eva	20	88.0	CSE	B
5	6	Bob	21	55.0	ECE	B

#4 from a student dataset,select students whose marks>60

```
above_60=df[df['Marks']>60]
print(above_60)
```

	Roll.no	Name	Age	Marks	Department	Section
0	1	Alice	20	78.0	CSE	A
2	3	Charlie	19	65.0	CSE	A
4	5	Eva	20	88.0	CSE	B

#5 sort the students based on marks in descending order

```
sorted_df=df.sort_values(by='Marks',ascending=False)
print(sorted_df)
```

	Roll.no	Name	Age	Marks	Department	Section
4	5	Eva	20	88.0	CSE	B
0	1	Alice	20	78.0	CSE	A
2	3	Charlie	19	65.0	CSE	A
5	6	Bob	21	55.0	ECE	B
3	4	David	22	45.0	EEE	C
1	2	Bob	21	0.0	ECE	B

#6 add a new column result that shows "pass" if marks>=50,else"fail"

```
df['Result']=df['Marks'].apply(lambda x:'pass' if x>=50 else 'fail')
print(df)
```

	Roll.no	Name	Age	Marks	Department	Section	Result
0	1	Alice	20	78.0	CSE	A	pass
1	2	Bob	21	0.0	ECE	B	fail
2	3	Charlie	19	65.0	CSE	A	pass
3	4	David	22	45.0	EEE	C	fail
4	5	Eva	20	88.0	CSE	B	pass
5	6	Bob	21	55.0	ECE	B	pass

```
#7 count the total number of students in the dataset
total_students=len(df)
print("total_students:",total_students)
```

```
total_students: 6
```

```
#8 find the average marks of all students
average_marks=df['Marks'].mean()
print("average_marks:",average_marks)
```

```
average_marks: 55.166666666666664
```

```
#9 remove duplicate student records from the dataset
df=df.drop_duplicates()
print(df)
```

	Roll.no	Name	Age	Marks	Department	Section	Result
0	1	Alice	20	78.0	CSE	A	pass
1	2	Bob	21	0.0	ECE	B	fail
2	3	Charlie	19	65.0	CSE	A	pass
3	4	David	22	45.0	EEE	C	fail
4	5	Eva	20	88.0	CSE	B	pass
5	6	Bob	21	55.0	ECE	B	pass

1.3)Matplotlib

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Step 1 Create sample dataset
data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
    'ExamScore': [35, 40, 50, 55, 65, 70, 78, 85]
}

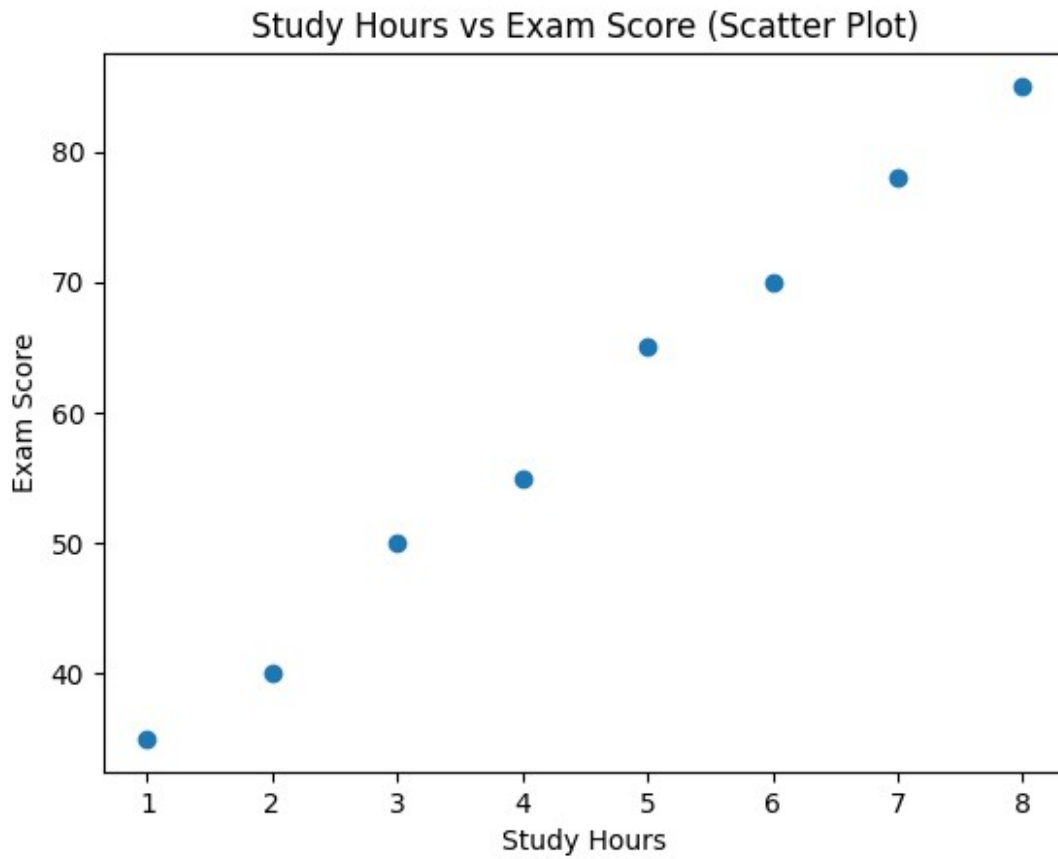
df = pd.DataFrame(data)
print(df)
```

	StudyHours	ExamScore
0	1	35
1	2	40
2	3	50
3	4	55
4	5	65
5	6	70
6	7	78
7	8	85

```
# Step 2 Ensure numeric columns (important!)
df['StudyHours'] = pd.to_numeric(df['StudyHours'], errors='coerce')
df['ExamScore'] = pd.to_numeric(df['ExamScore'], errors='coerce')

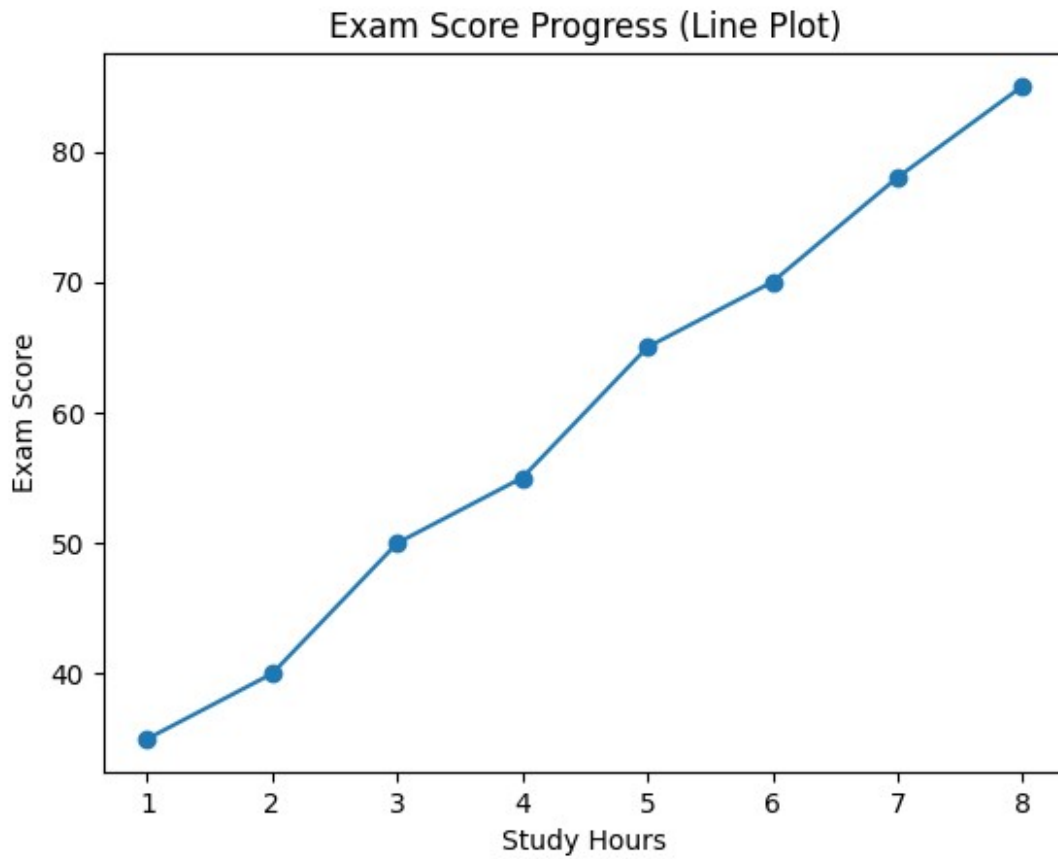
# Step 3 Drop rows with missing values (if any)
df = df.dropna(subset=['StudyHours', 'ExamScore'])

# 1 Scatter Plot
plt.scatter(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Scatter Plot)")
plt.show()
```

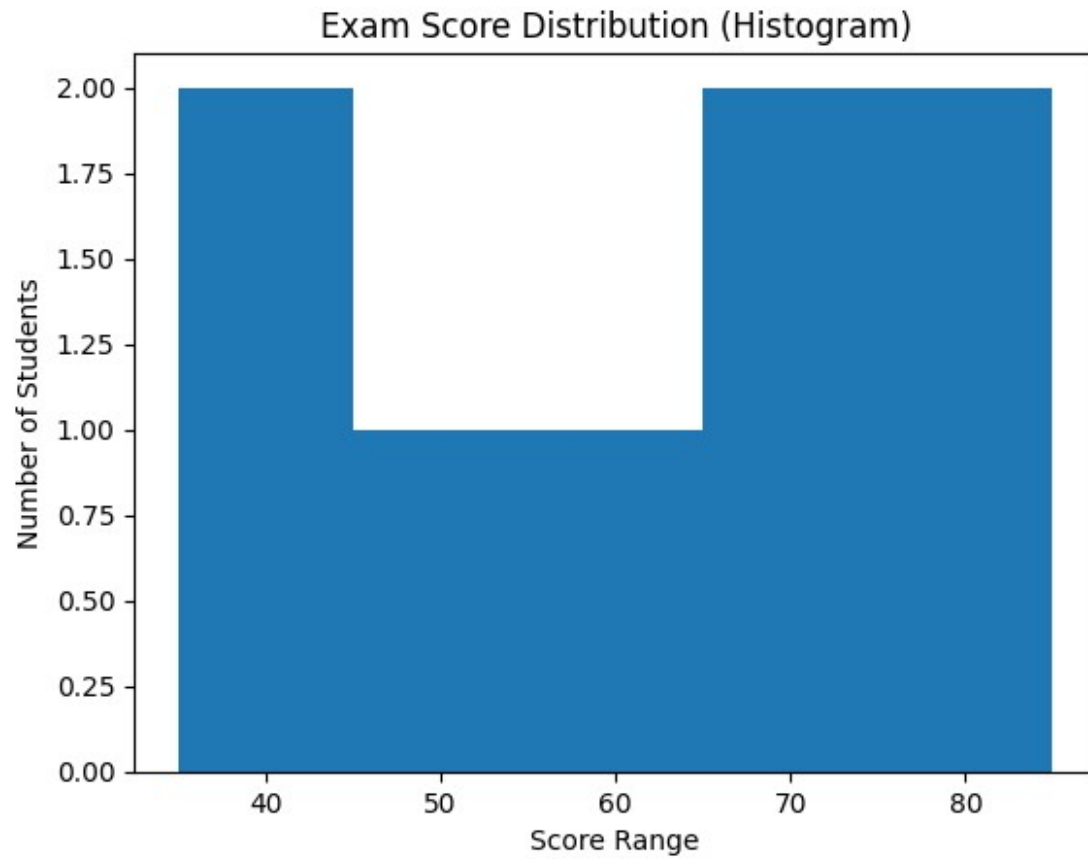


2 Line Plot

```
plt.plot(df['StudyHours'], df['ExamScore'], marker='o')  
plt.xlabel("Study Hours")  
plt.ylabel("Exam Score")  
plt.title("Exam Score Progress (Line Plot)")  
plt.show()
```

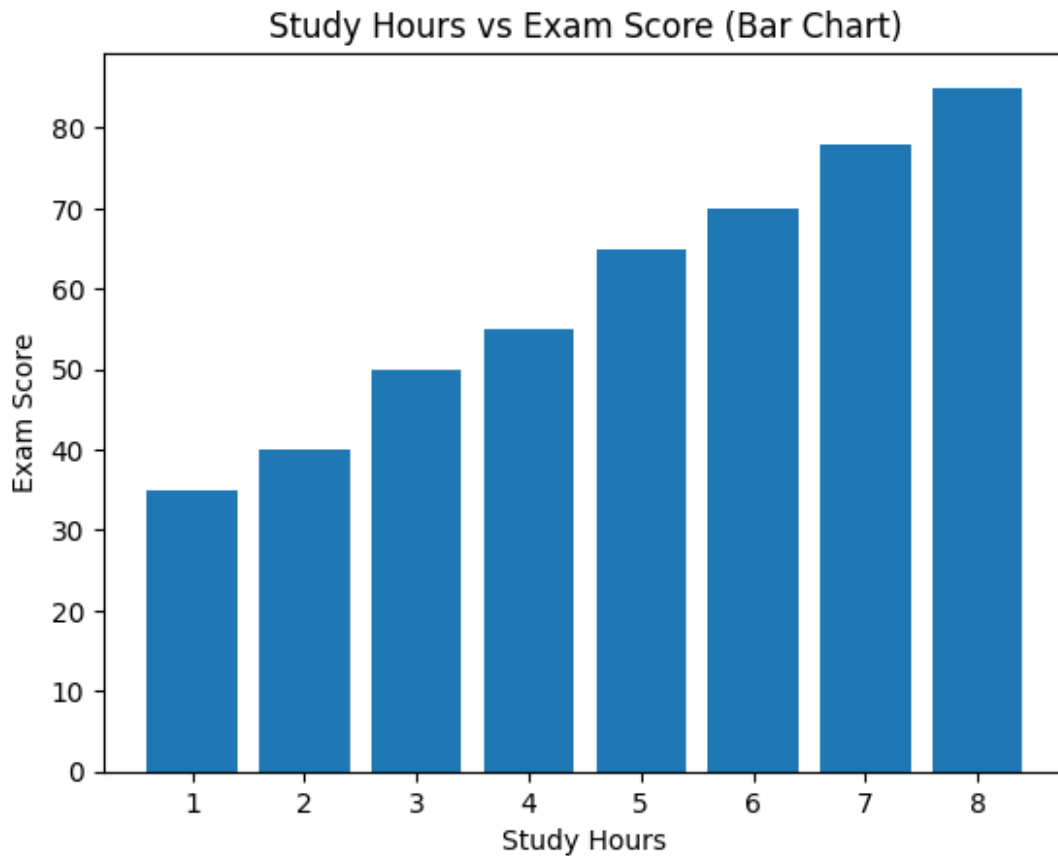


```
# 3 Histogram
plt.hist(df['ExamScore'], bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution (Histogram)")
plt.show()
```



4 Bar Chart

```
plt.bar(df['StudyHours'], df['ExamScore'])  
plt.xlabel("Study Hours")  
plt.ylabel("Exam Score")  
plt.title("Study Hours vs Exam Score (Bar Chart)")  
plt.show()
```



Student management system scenario based questions

Code after importing CSV file(dataset for pandas(11249a381) - Sheet1.csv

```
#10 draw the scatter plot for ant two columns
```

```
 #(age vs marks)
```

```
# Step 1 Load the CSV file
```

```
df = pd.read_csv("/content/dataset for pandas(11249a381) - Sheet1.csv")
```

```
# Step 2 Ensure numeric columns (convert strings to numbers, invalid values become NaN)
```

```
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
```

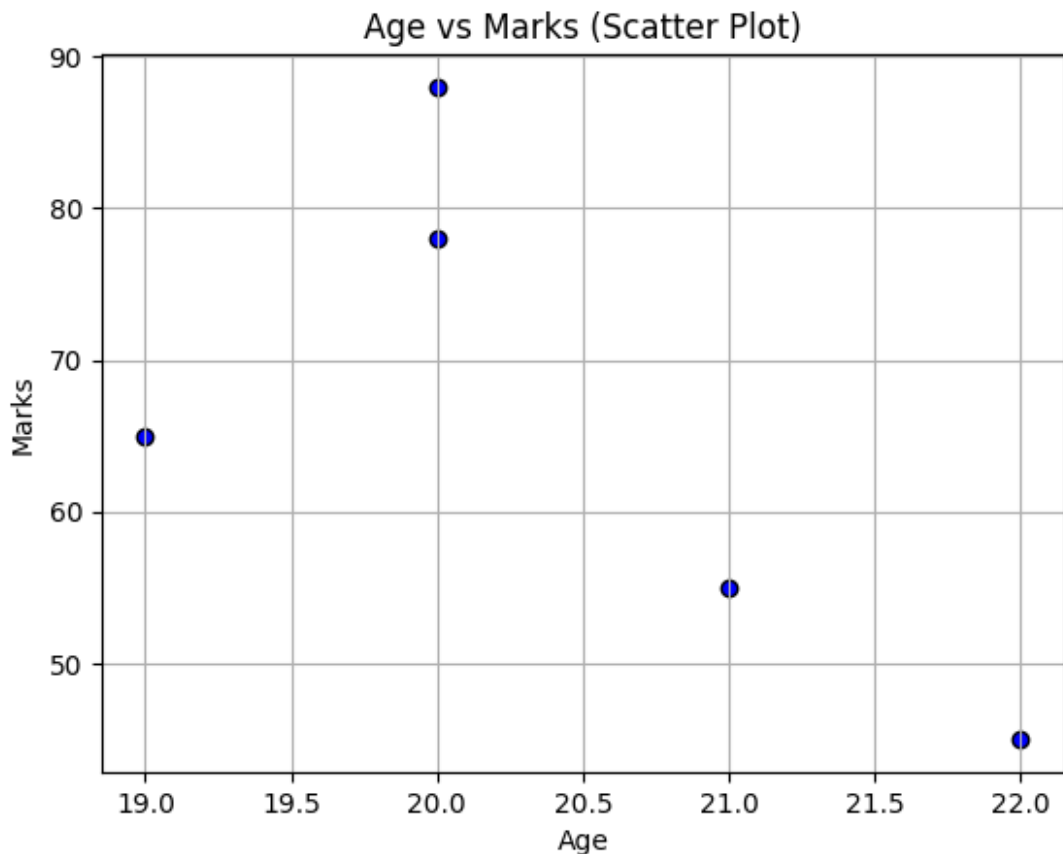
```
df['Marks'] = pd.to_numeric(df['Marks'], errors='coerce')
```

```
# Step 3 Drop rows with missing Age or Marks
```

```
df_clean = df.dropna(subset=['Age', 'Marks'])
```

```
# Step 4 Draw Scatter Plot
```

```
plt.scatter(df_clean['Age'], df_clean['Marks'], color='blue',
            edgecolor='black')
plt.xlabel("Age")
plt.ylabel("Marks")
plt.title("Age vs Marks (Scatter Plot)")
plt.grid(True)  # Optional: shows grid lines
plt.show()
```



```
#11 draw the Bar chart for marks
import matplotlib.pyplot as plt

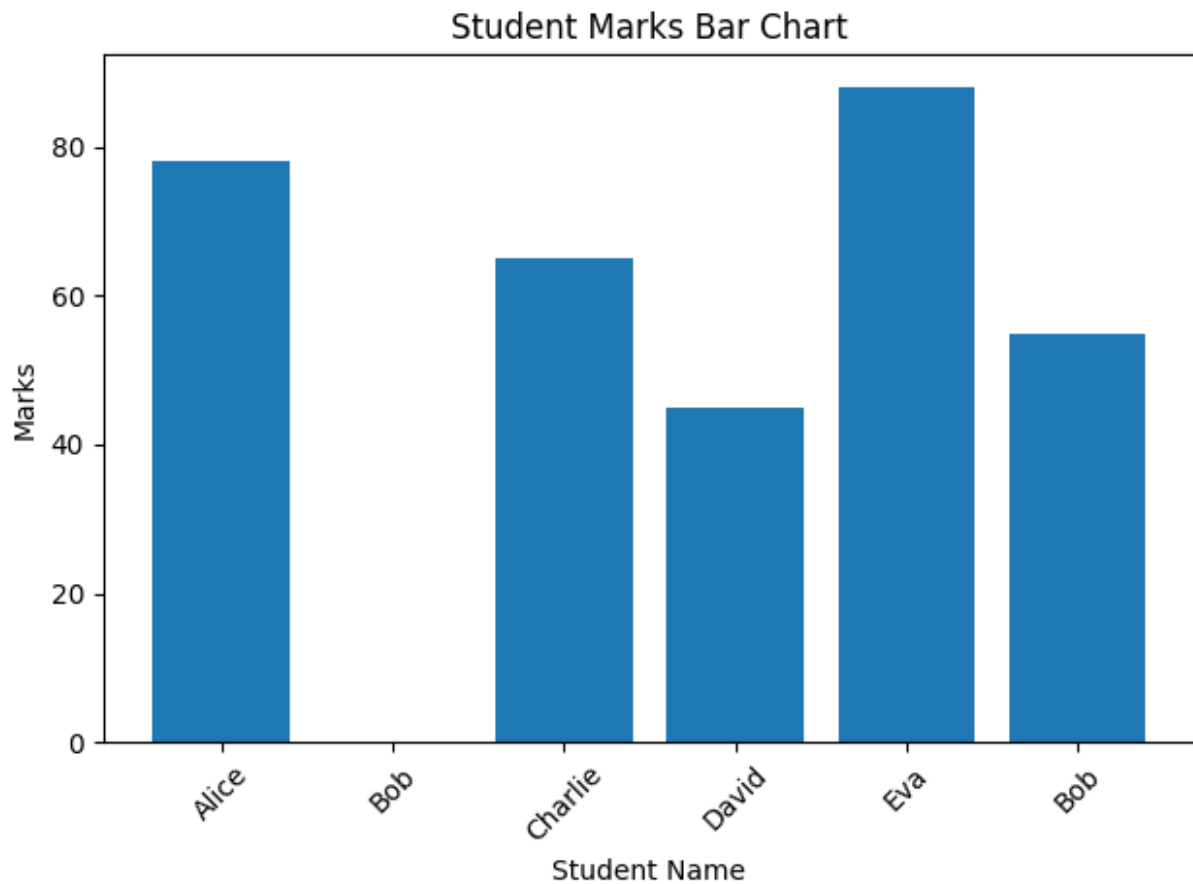
# Ensure 'Marks' is numeric and fill missing values
df['Marks'] = pd.to_numeric(df['Marks'], errors='coerce').fillna(0)

# Remove duplicate rows
df = df.drop_duplicates()

# Use the correct column name for x-axis
x_column = 'Roll.no'
y_column = 'Marks'

# Draw bar chart
plt.bar(df[x_column], df[y_column])
```

```
plt.xlabel("Student Name")
plt.ylabel("Marks")
plt.title("Student Marks Bar Chart")
plt.xticks(df[x_column], df['Name'], rotation=45) # show names
instead of roll numbers
plt.tight_layout() # prevents label cutoff
plt.show()
```



#STEP 1:IMPORT LIBRARIES

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

#STEP 2:CREATE DATASET

```
data = {
    'Student': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
```

	Student	Classes_Attended	Internal_Marks
0	S1	30	35
1	S2	35	38
2	S3	40	42
3	S4	45	46
4	S5	50	50
5	S6	55	55
6	S7	60	60
7	S8	65	65
8	S9	70	70
9	S10	75	75

STEP 3: FEATURES & TARGET

```
X = df[['Classes_Attended']]
y = df['Internal_Marks']
```

STEP 4: TRAIN-TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7
)
```

STEP 5: TRAIN MODEL

```
model = LinearRegression()  
model.fit(X_train, y_train)  
  
LinearRegression()
```

STEP 6: PREDICTION

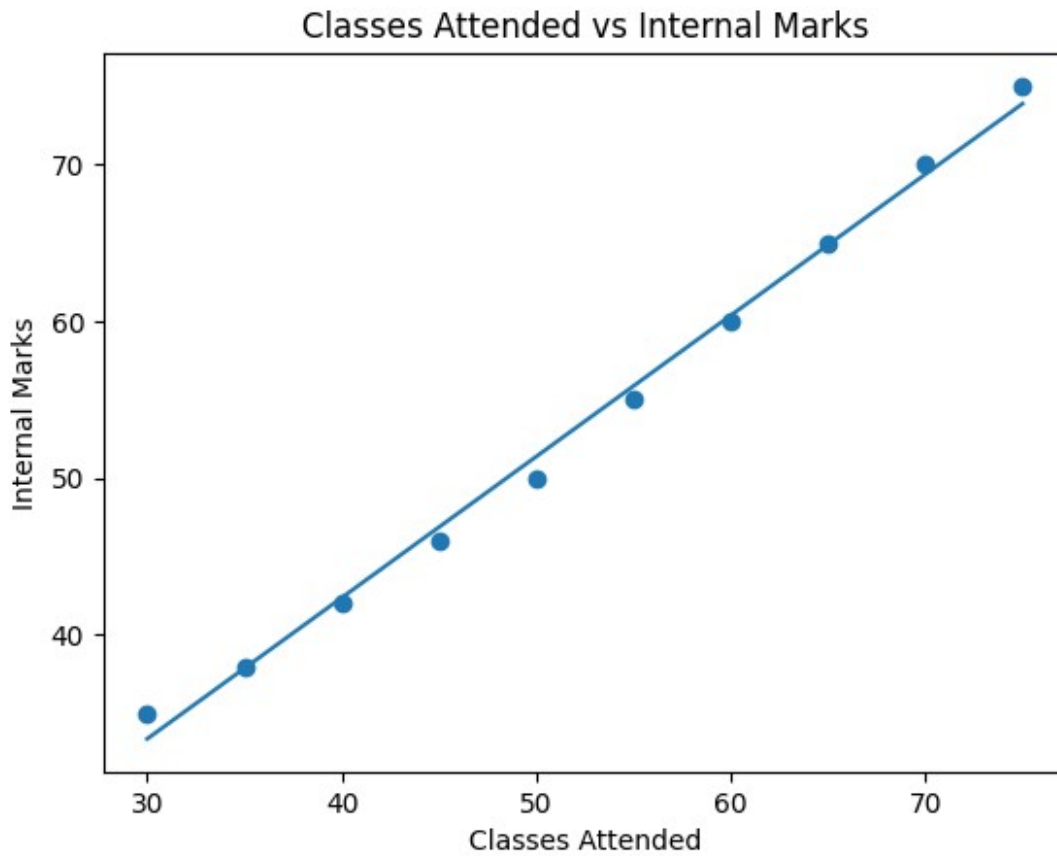
```
predictions = model.predict(X_test)
```

STEP 7: EVALUATION

```
mse = mean_squared_error(y_test, predictions)  
print("MSE:", mse)  
print("Marks per Class:", model.coef_[0])  
print("Base Marks:", model.intercept_)  
  
MSE: 0.578125  
Marks per Class: 0.9  
Base Marks: 6.375
```

STEP 8: VISUALIZATION

```
plt.scatter(X, y)  
plt.plot(X, model.predict(X))  
plt.xlabel("Classes Attended")  
plt.ylabel("Internal Marks")  
plt.title("Classes Attended vs Internal Marks")  
plt.show()
```



Result:

Successfully explored and implemented basic operations using NumPy arrays, performed data handling using Pandas, visualized data using Matplotlib, and applied simple machine learning models using Scikit-learn.

#2. Perform Data exploration and preprocessing in Python

Aim:

To perform data exploration and preprocessing techniques such as handling missing values, encoding categorical variables, normalization, and feature scaling.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
# --- STEP 1: LOAD DATA (Creating a dummy dataset for the lab) ---
data = {
    'Country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35], # Note the 'nan' (missing value)
    'Salary': [40000, 60000, 50000, np.nan, 72000, 58000],
    'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']
}
df = pd.DataFrame(data)
print("--- ORIGINAL RAW DATA ---")
print(df)
print("\n")
# --- STEP 2: INSPECT DATA ---
# Check for missing values
print("--- MISSING VALUES COUNT ---")
print(df.isnull().sum())
print("\n")
# --- STEP 3: CLEAN DATA (Handling Missing Values) ---
# Logic: Fill missing Age/Salary with the Average (Mean) of that column
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
print("--- DATA AFTER CLEANING ---")
print(df)
print("\n")
# --- STEP 4: CONVERT TEXT TO NUMBERS (Encoding) ---
# Logic: Machines can't read 'India' or 'USA'. We convert them to 0s and 1s.
# We use 'One Hot Encoding' (get_dummies)
df_encoded = pd.get_dummies(df, columns=['Country'])
# For the Target column (Purchased), let's map Yes/No manually
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})
print("--- DATA AFTER ENCODING (All Numbers Now) ---")
print(df_encoded)
print("\n")
# --- STEP 5: SCALE FEATURES ---
# Logic: Age is 20-30, Salary is 40000-60000. Salary dominates Age because it's bigger.
```

```
# We shrink them to the same scale.
# Separate Features (X) and Target (y)
X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']
# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("--- FINAL PROCESSED DATA (Ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

--- ORIGINAL RAW DATA ---

	Country	Age	Salary	Purchased
0	India	22.0	40000.0	No
1	USA	25.0	60000.0	Yes
2	India	NaN	50000.0	Yes
3	USA	30.0	NaN	No
4	UK	28.0	72000.0	Yes
5	India	35.0	58000.0	Yes

--- MISSING VALUES COUNT ---

Country	0
Age	1
Salary	1
Purchased	0

dtype: int64

--- DATA AFTER CLEANING ---

	Country	Age	Salary	Purchased
0	India	22.0	40000.0	No
1	USA	25.0	60000.0	Yes
2	India	28.0	50000.0	Yes
3	USA	30.0	56000.0	No
4	UK	28.0	72000.0	Yes
5	India	35.0	58000.0	Yes

--- DATA AFTER ENCODING (All Numbers Now) ---

	Age	Salary	Purchased	Country_India	Country_UK	Country_USA
0	22.0	40000.0	0	True	False	False
1	25.0	60000.0	1	False	False	True
2	28.0	50000.0	1	True	False	False
3	30.0	56000.0	0	False	False	True
4	28.0	72000.0	1	False	True	False
5	35.0	58000.0	1	True	False	False

--- FINAL PROCESSED DATA (Ready for AI) ---

Age	Salary	Country_India	Country_UK	Country_USA
-----	--------	---------------	------------	-------------

0	-1.484615	-1.644453	1.0	-0.447214	-0.707107
1	-0.742307	0.411113	-1.0	-0.447214	1.414214
2	0.000000	-0.616670	1.0	-0.447214	-0.707107
3	0.494872	0.000000	-1.0	-0.447214	1.414214
4	0.000000	1.644453	-1.0	2.236068	-0.707107

Result: Successfully analyzed the dataset, handled missing values, transformed categorical data, and prepared a clean dataset suitable for machine learning models.

#EXPERIMENT 3,Implement Regularized Linear Regression

Aim:

To implement linear regression with regularization techniques (Ridge and Lasso) to prevent overfitting.

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn import datasets
from sklearn.model_selection import train_test_split

diabetes = datasets.load_diabetes()
x_train, x_test, y_train, y_test = train_test_split(diabetes.data,
diabetes.target, test_size=0.2, random_state=42)

lr = LinearRegression().fit(x_train, y_train)
ridge = Ridge(alpha=1.0).fit(x_train, y_train)
lasso = Lasso(alpha=1.0).fit(x_train, y_train)

print(f"Linear Regression MSE: {mean_squared_error(y_test,
lr.predict(x_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test,
ridge.predict(x_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test,
lasso.predict(x_test)):.2f}")

Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 3403.58
```

Result:

Successfully built a regularized linear regression model and observed improved generalization performance by reducing overfitting.

Aim:

To implement the Naive Bayes classification algorithm using a dataset stored in a CSV file.

A Naive Bayes classifier is a supervised, probabilistic machine learning algorithm used for classification tasks, particularly in high-dimensional datasets like text analysis. It applies Bayes' Theorem with a "naive" assumption that all input features are independent of each other. It is highly efficient for spam filtering, sentiment analysis, and, document classification.

Key Aspects of Naive Bayes:

Bayes' Theorem: It calculates the probability of a class based on prior knowledge of conditions related to the class.

Bayes Theorem Tells,

What is the probability of something being true AFTER seeing some evidence?

Formula

$P(A) \rightarrow \text{Prior}$

What we believed before seeing evidence

$P(B|A) \rightarrow \text{Likelihood}$

How likely is the evidence if A is true?

$P(A|B) \rightarrow \text{Posterior}$

Updated probability after seeing evidence

$P(B) \rightarrow \text{Normalizing factor (just ensures probability stays valid)}$

Before evidence: 70% pass rate.

After seeing strong study evidence: Now probability increased to 90%.

It updates beliefs using evidence.

Common Variants:

- o Gaussian Naive Bayes: Used for continuous data that follows a normal distribution.
- o Multinomial Naive Bayes: Primarily used for document classification and word counts.
- o Bernoulli Naive Bayes: Used when features are binary (e.g., word exists or does not exist).

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

Create dataset manually

```
data = {
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['pass', 'fail', 'pass', 'fail', 'pass', 'fail', 'pass', 'fail']
}
df = pd.DataFrame(data)
```

Features and target

```
x = df[['StudyHours', 'Attendance']]
y = df['Result']
```

Split data into training and testing

```
x_train, x_test, y_train, y_test =
train_test_split(x, y, test_size=0.25, random_state=42)
```

Train model

```
model = GaussianNB()
model.fit(x_train, y_train)

GaussianNB()
```

Make predictions

```
y_pred = model.predict(x_test)
```

calculate accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("predicted results:", y_pred)
print("actual results:", y_test.values)
print("Accuracy:", accuracy)
```

```
predicted results: ['fail' 'fail']  
actual results: ['fail' 'fail']  
Accuracy: 1.0
```

Result:

Successfully trained and tested the Naive Bayes model, achieving accurate classification based on probability principles.

Aim:

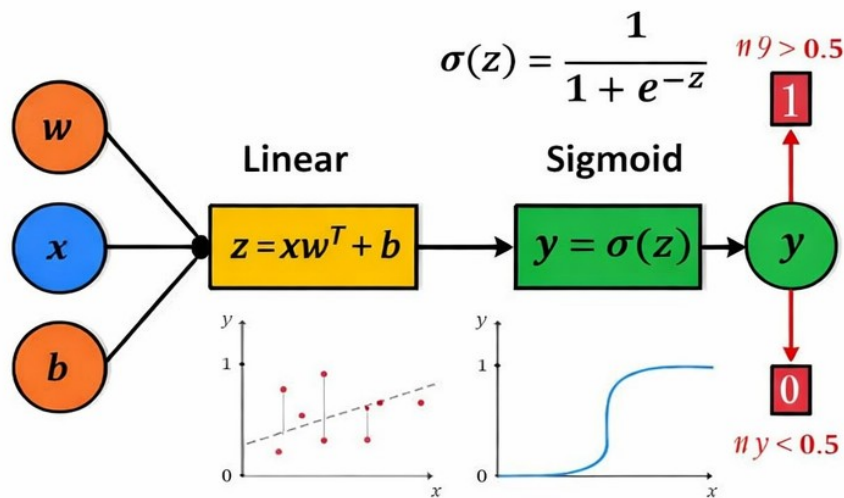
To implement logistic regression with regularization to improve classification performance and avoid overfitting.

Ex No.5.) Implement regularized logistic regression

Logistic Regression:

Logistic Regression is a supervised classification algorithm used to predict categorical outcomes (mostly binary: Yes/No, 0/1, True/False).

Even though its name contains regression, it is mainly used for classification problems.



In your code, the dataset is Breast Cancer Detection from Scikit-learn

Target:

- 0 → Malignant
- 1 → Benign

Code:

1.import libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

2.Load datasets

```
data = load_breast_cancer()
X = data.data
y = data.target
```

3.Split

```
X_train,X_test,y_train,y_test =
train_test_split(X,y,test_size=0.2,random_state=42)
```

4.Scaling

```
scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
```

5.Normal Logistic Regression (very Large C)

```
normal_model=LogisticRegression(C=1e6,penalty='l2',solver='liblinear')
normal_model.fit(X_train,y_train)

normal_pred=normal_model.predict(X_test_scaled)
normal_acc=accuracy_score(y_test,normal_pred)
```

6.Regularized Logistic Regression

```
regularized_model=LogisticRegression(C=1.0,penalty='l2',solver='liblinear')
regularized_model.fit(X_train,y_train)

reg_pred=regularized_model.predict(X_test_scaled)
reg_acc=accuracy_score(y_test,reg_pred)
```

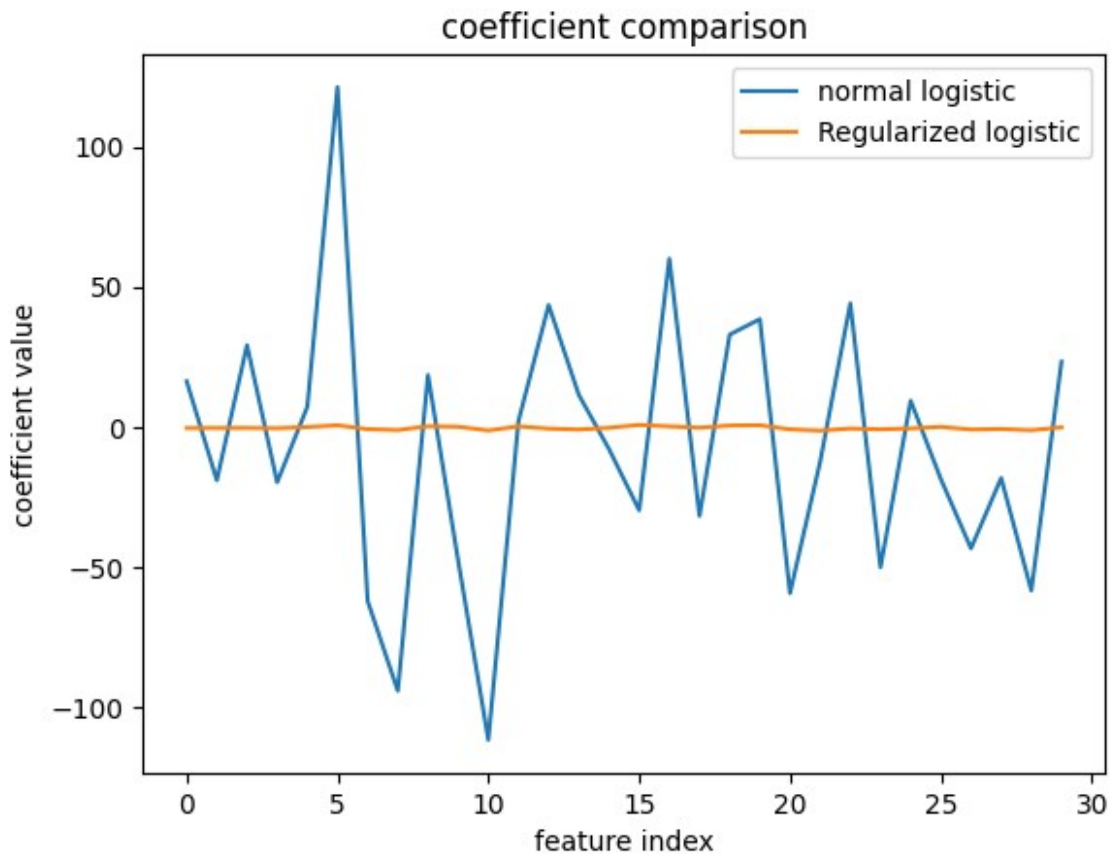
7.print results

```
print("Normal Logistic Regression Accuracy:",normal_acc*100)
print("Regularized Logistic Regression Accuracy:",reg_acc*100)
```

Normal Logistic Regression Accuracy: 93.85964912280701
Regularized Logistic Regression Accuracy: 97.36842105263158

8.compare coefficients

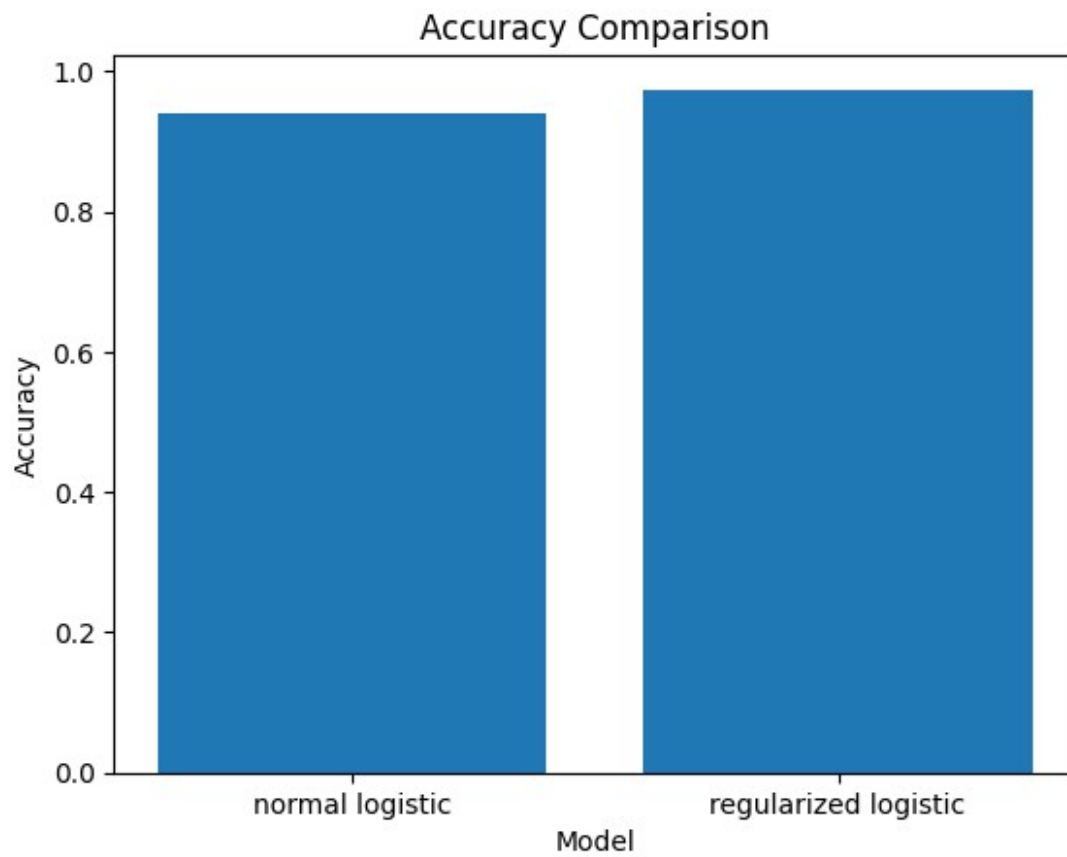
```
plt.figure()
plt.plot(normal_model.coef_.flatten(),label='normal logistic')
plt.plot(regularized_model.coef_.ravel(),label='Regularized logistic')
plt.legend()
plt.title("coefficient comparison")
plt.xlabel("feature index")
plt.ylabel("coefficient value")
plt.show()
```



9.Accuracy Comparison Bar Graph

```
plt.figure()
models=['normal logistic','regularized logistic']
accuracies = [normal_acc,reg_acc]
```

```
plt.bar(models, accuracies)
plt.title("Accuracy Comparison")
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.show()
```



Result: Successfully developed a logistic regression model with regularization and achieved better classification accuracy on test data.

6. Build Models using Different Ensembling Techniques

Aim:

To build machine learning models using ensembling techniques such as Bagging, Boosting, and Random Forest.

Objective

To implement and compare Bagging, Boosting, and Stacking.

```
from sklearn.ensemble import RandomForestClassifier,
AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data,
iris.target,
test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [('rf', RandomForestClassifier()), ('svc',
SVC(probability=True))]
stack = StackingClassifier(estimators=estimators,
final_estimator=LogisticRegression()).fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test,
rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test,
ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test,
stack.predict(X_test)):.2f}")
```

Random Forest Accuracy: 1.00

AdaBoost Accuracy: 0.93

Stacking Accuracy: 1.00

Result:

Successfully implemented ensemble models and observed improved accuracy and robustness compared to individual models.

7. Build Models using Decision Trees

Aim:

To build a classification/regression model using Decision Tree algorithms.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data,
                                                    iris.target,
                                                    test_size=0.2, random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
print (f"Decision Tree Classifier Accuracy: {accuracy_score(y_test,
dt_clf.predict(X_test)):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

Decision Tree Classifier Accuracy: 1.00

Feature Importances: [0. 0. 0.39426873 0.60573127]

Result:

Successfully constructed a Decision Tree model and visualized decision rules for prediction.

8) Support Vector Machine (SVM) with Different Kernels

Support Vector Machine is a supervised machine learning algorithm used for classification and regression problems.

SVM finds the best hyperplane that separates data points of different classes.

Example:

Class A..

Class B AAA

SVM finds a line (in 2D) or hyperplane (in higher dimensions) that separates these classes with maximum margin.

Key terms:

Hyperplane

Decision boundary separating classes.

Support Vectors

The data points closest to the decision boundary.

Margin

Distance between hyperplane and nearest data points.

Kernel Functions

Sometimes data is not linearly separable. Kernels transform data into a higher dimension to separate classes.

Common kernels:

1 Linear Kernel

Used when data is linearly separable.

2 Polynomial Kernel

Creates curved decision boundaries.

3 RBF Kernel (Radial Basis Function)

Most commonly used kernel.

Algorithm Steps

1 Load dataset

2 Split dataset into training and testing

3 Train SVM model with different kernels

4 Predict output

5 Compare accuracy

Aim:

To implement Support Vector Machine models using different kernels (linear, polynomial, RBF).

```
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
```

```

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

# Linear Kernel
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)

# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)

# RBF Kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)

```

```

Linear Kernel Accuracy: 0.9666666666666667
Polynomial Kernel Accuracy: 0.9333333333333333
RBF Kernel Accuracy: 0.9666666666666667

```

Result:
Successfully trained SVM models with various kernels and compared their performance on the dataset

9) Implement K-Nearest Neighbors (KNN) Classification

Aim:

To implement the K-NN algorithm for classification of a dataset.

To implement the K-Nearest Neighbors (KNN) algorithm to classify mobile phones into price categories using Python.

Dataset:

Mobile Price Classification Dataset. It classifies mobile phones into 4 price ranges based on features like RAM, battery power, etc.

Kaggle Dataset:

<https://www.kaggle.com/datasets/iabhishekofficial/mobile-price-classification>

This dataset is good because:

Only numerical features (easy for KNN)

Multi-class classification

Clean CSV dataset suitable for engineering lab experiments

Algorithm

1. Import required libraries.
2. Load the dataset.
3. Split dataset into features (X) and target (y).
4. Divide data into training and testing sets.
5. Normalize features using StandardScaler.
6. Train the KNN classifier.
7. Predict results for test data.
8. Evaluate accuracy.

Step 1: Import libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
import numpy as np # Import numpy for random number generation
```

Step 2: Load dataset

```
try:
    data = pd.read_csv("train.csv")
except FileNotFoundError:
    try:
        data = pd.read_csv("test.csv")
    except FileNotFoundError:
        print("Error: Neither train.csv nor test.csv found.")
```

```

        # You might want to exit or raise the error here if both are
missing
        raise # Re-raise the error if both files are missing

# Step 3: Define features and target
if "price_range" in data.columns:
    X = data.drop("price_range", axis=1)
    y = data["price_range"]
else:
    # Generate a dummy 'price_range' column with random values (0, 1,
2, 3)
    data['price_range'] = np.random.randint(0, 4, size=len(data))
    X = data.drop("price_range", axis=1)
    y = data["price_range"]

# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)

# Step 5: Feature scaling
syn_scaler = StandardScaler()
X_train = syn_scaler.fit_transform(X_train)
X_test = syn_scaler.transform(X_test)

# Step 6: Create KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# Step 7: Train model
knn.fit(X_train, y_train)

#Step 8: Prediction
y_pred = knn.predict(X_test)

# Step 9: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))

```

Accuracy: 0.195

Confusion Matrix:

```

[[ 9 20 11  8]
 [14 15  5 10]
 [19 13  9  8]
 [17 20 16  6]]

```

Classification Report:

	precision	recall	f1-score	support
0	0.15	0.19	0.17	48
1	0.22	0.34	0.27	44

	2	0.22	0.18	0.20	49
	3	0.19	0.10	0.13	59
accuracy				0.20	200
macro avg		0.20	0.20	0.19	200
weighted avg		0.19	0.20	0.19	200
Result:					
Successfully classified data using the K-NN algorithm and analyzed performance based on different values of K.					

10. Clustering using K-Means with PCA and Elbow Method

Aim:

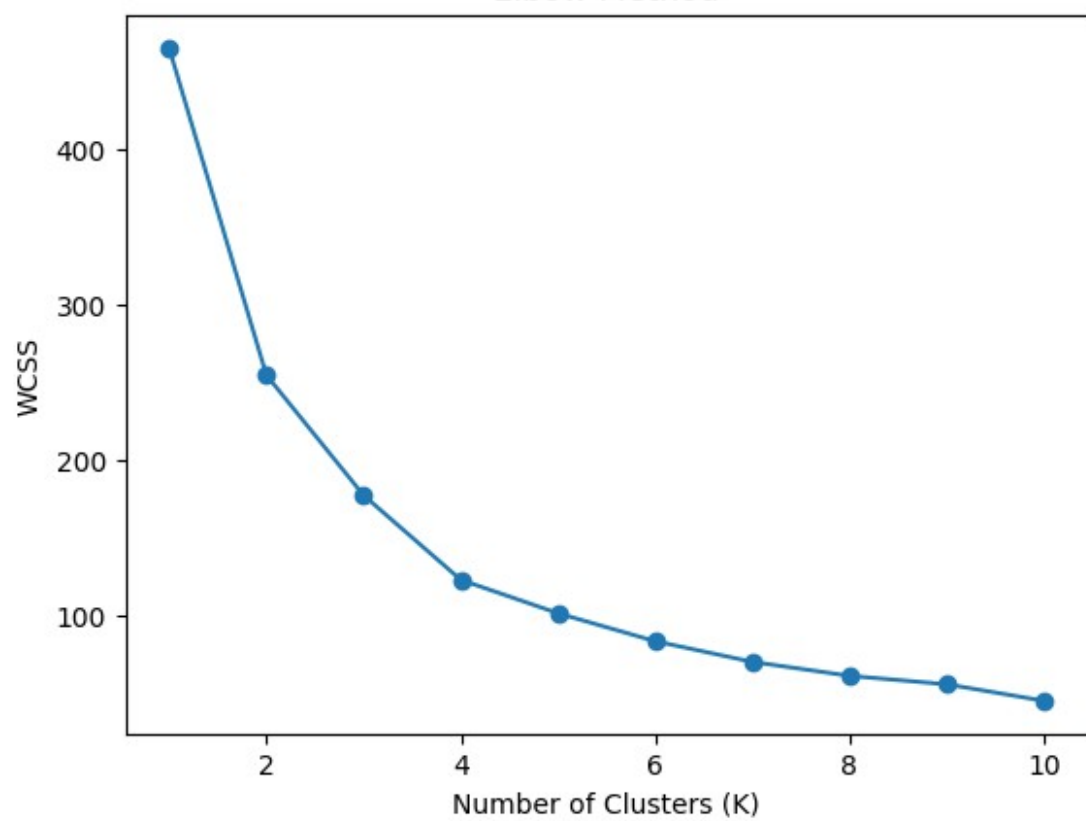
To perform clustering using K-Means after reducing dimensionality using PCA and determining optimal K using the Elbow method.

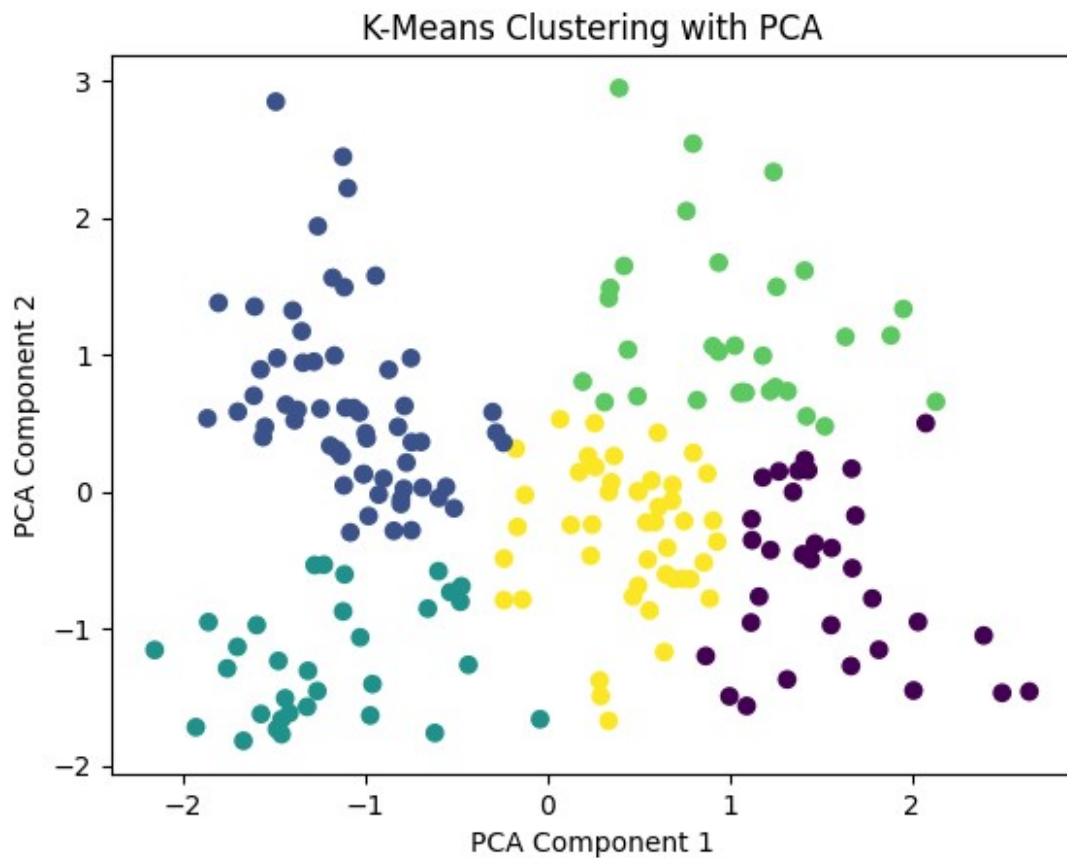
```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv("Mall_Customers.csv")
# Select numerical features
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X);
# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
# Elbow Method
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=0, n_init='auto')
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
# Plot Elbow Graph
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
# Apply KMeans with optimal K (example K=5)
kmeans = KMeans(n_clusters=5, random_state=0, n_init='auto')
clusters = kmeans.fit_predict(X_pca)
# Plot Clusters
plt.scatter(X_pca[:,0], X_pca[:, 1], c=clusters)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

Explained Variance Ratio: [0.44266167 0.33308378]

Elbow Method





Result:

Successfully reduced dataset dimensions using PCA, identified optimal clusters using the Elbow method, and performed K-Means clustering effectively.